IRE Assignment 2 — Final Report

Learning from Click-Logs on MIND and EB-NeRD

Neill Rawani (2025202027) · Sukhraj Singh (2025202009)
Team: The Suitors · Codabench: `apollo19`

Code: `https://github.com/sukhraj01/News-Recommendation-Retrieval/tree/a2-final-polish`

September 26, 2026

We built a news recommendation system out of user behaviour and click impressions on two datasets in different languages, namely MIND (English) and EB-NeRD (Danish). The user impressions were the core part of our system and we extracted all relevant data from them. This assignment is a continuation of Assignment 1, which used the same two datasets.

1 Assignment Part 1, and What We Added

The lexical and semantic approaches were built in Assignment 1, where data from both datasets was stored in a unified schema and the official temporal splits were used as-is. For lexical retrieval, articles were ranked by the BM25 algorithm using term frequencies and inverse document frequencies, with saturation and document-length normalisation. For semantic retrieval, each article's title and abstract was encoded into a 384-dimensional vector by a multilingual sentence-transformer, a user was represented by the mean of the vectors of the articles they had clicked, and candidates were ranked by cosine similarity to that user vector.

Beyond what Assignment 1 required, this phase also produced the 65 engineered behavioural features behind our EB-NeRD ranking model, which remain the strongest EB-NeRD system we have. MIND had no equivalent feature set at this point.

To build on this, we added a behaviour layer: we engineered a two-stage retrieve-and-rank pipeline and click-history features, reproduced each dataset's official NRMS baseline to measure it honestly, and then tried to improve on it with one principled change per dataset, testing each claimed gain with a paired bootstrap confidence interval.

Our final submissions on the Codabench leaderboards scored **0.6766 on MIND** (submission 930992) and **0.7542 on EB-NeRD** (submission 907863). Both leaderboards report a Score column whose value matches our own locally measured AUC closely, so we read Score as AUC, though the column headers are not visible in our screenshots. The screenshots for both submissions are in the repository under `results/leaderboard_screenshots/`.

2 Q1: Click-History and Session Features

The two datasets got different feature sets, because they carry different information. EB-NeRD has 65 engineered features covering long-term and short-term category, topic and embedding affinity, interest drift, user reading habits, article properties, impression context, the article the user is reading at that moment, session position, and seven features that normalise a candidate against the others shown in the same impression. For MIND we added 5 new features in this assignment: click count, category match, subcategory match, and two recency-weighted category and subcategory affinities using a geometric decay of 0.9.

Among the EB-NeRD features, freshness is responsible for 33.3% of the model's gain, led by `article_age_h`, which is the single top feature by gain throughout. Long-term interest contributes

14.1%, while short-term interest contributes only 0.8%, which falsified the hypothesis this feature set was originally built to test. What actually decides a click on EB-NeRD is which candidate is freshest relative to the others on offer, and what the user is reading right now. Every feature is restricted to information available before the impression: EB-NeRD's click history is a 21-day window that closes strictly before the impression window opens, and an integration test asserts that no user's history click falls at or after their own earliest impression.

Results for Q1

On EB-NeRD, the 65 features feed a LightGBM learning-to-rank model that scores AUC 0.7588 on the `ebnerd_small` validation split and 0.7542 on the real leaderboard.

On MIND, the 5 new features were combined by a logistic regression and scored AUC 0.6176 (95% CI 0.6153–0.6197) against the deployed embedding baseline's 0.6340, a paired difference of −0.0164 (95% CI −0.0191 to −0.0139). MRR, nDCG@5 and nDCG@10 all lost by the same margin. Also, the confidence interval was roughly ten times narrower than the gap, so tuning would not have closed it. The features were not adopted.

What We Learnt

Click count turned out to contribute nothing to ranking, as it depends only on the user, so it is identical for every candidate within an impression, and AUC, MRR and nDCG depend only on the order of candidates inside an impression. Adding the same constant to every score cannot change that order. We verified this by zeroing the feature and re-scoring every dev impression: the maximum absolute change in AUC was exactly 0.00e+00.

MIND lacks freshness and session data entirely, which is why the EB-NeRD recipe could not simply be carried across. We confirmed this by direct measurement: `article_published_time` is null for all 72,023 MINDlarge articles, and `session_id`, `dwell_time`, `scroll_percentage` and `is_front_page` are null for all 14,085,557 impression rows. MIND's source files never contain these fields, so this is a dataset limitation and not something feature engineering can work around.

3 Q2: The Re-Ranking

Our BM25 retriever fetches the top 200 articles out of the whole corpus, using the user's click history as the query, and the re-ranker then reorders those 200. The re-ranker is the trained NRMS model from Q3, which encodes each candidate's text and the user's history into vectors and scores a candidate as the dot product of the two; we used the fully trained MIND checkpoint (10 epochs on MINDlarge train, dev AUC 0.6868) because an earlier check with a one-epoch model measured nothing and only showed that an undertrained model reorders badly.

Metric	Before (BM25 order)	After (NRMS re-rank)	Paired difference (95% CI)
AUC	0.5428	**0.7224**	+0.1796 [+0.1375, +0.2213]
MRR			+0.0034 [+0.0022, +0.0045]
nDCG@5			+0.1113 [+0.0757, +0.1483]
nDCG@10			+0.1359 [+0.1016, +0.1740]

8,000 sampled MINDlarge-dev impressions; hit rate (recall@200) 3.21% (2.83–3.61%). Before/after metrics cover the 257 impressions whose retrieved top-200 contained the clicked article.

Evaluated on 8,000 sampled MINDlarge-dev impressions, re-ranking lifted AUC from 0.5428 in raw BM25 order to 0.7224, a paired difference of +0.1796 (95% CI +0.1375 to +0.2213). nDCG@5 improved by +0.1113 and nDCG@10 by +0.1359, all with confidence intervals excluding zero.

Retrieval becomes the bottleneck, because on average the top 200 contains the clicked article in only 3.21% of impressions (95% CI 2.83% to 3.61%), so roughly 97% of clicks never reach the re-ranker at all and no re-ranking model can recover them. The before-and-after numbers above are therefore

measured on the 257 impressions where retrieval did succeed. Both halves are the honest answer: re-ranking is a large, clear win when retrieval works, and it cannot compensate for retrieval's own miss rate.

We deliberately used NRMS and not our EB-NeRD LightGBM model as the re-ranker here. That model's features are impression-conditional, such as a candidate's position in the list shown to the user and its rank against the other candidates in that same impression, and an article retrieved from the corpus was never shown, so those values do not exist for it. We considered building a reduced, retrieval-safe version but rejected it, because it would have measured a different model's ceiling rather than ours.

4 Q3: Reproducing and Beating the Baseline

The official baseline for both datasets is NRMS, Neural News Recommendation with Multi-Head Self-Attention (Wu et al., 2019). It is the benchmark because both dataset authors ship an implementation of it and publish its scores. NRMS encodes each article's text with self-attention, builds a user vector the same way from the articles they clicked, and scores a candidate as the dot product of the two.

We first tried to run the official TensorFlow code on Ada, IIIT Hyderabad's GPU cluster, and gave ourselves one session to get it working. We could not build a working TensorFlow and CUDA environment in that time, mainly because downloads to the cluster ran at well under 1 MB/s. We then fell back to the alternative we had agreed on beforehand: a PyTorch reimplementation of the same model, written to match the official code layer by layer, using the hyperparameters from the official configuration files without changes. We describe it as a faithful reimplementation throughout the report, since we never ran the original code ourselves.

Both models were trained on an RTX 2080 Ti. The MIND control took just over 13 hours and scored AUC 0.6831 across all 376,471 MINDlarge dev impressions. Two things tell us the reproduction is trustworthy: it lands close to the 0.6776 the MIND paper reports, and it beats Candidate J, our own neural model from Assignment 1, by +0.0252 AUC on the same split. The EB-NeRD control scored AUC 0.5613 across all 244,647 `ebnerd_small` validation impressions, far below our LightGBM model's 0.7588 on that split. That gap is itself a finding: a text-only model has very little to separate candidates with on EB-NeRD.

We then made exactly one change per dataset, so any gain could be attributed to that change alone. For EB-NeRD we added the candidate's age at the time it was shown, because our feature-importance analysis had shown freshness carries 33.3% of the LightGBM model's gain. For MIND we gave the model more article text, adding the abstract to the title, motivated by the MIND paper's own results showing richer news representations score higher. We had originally planned to test GloVe initialisation against random initialisation on MIND and dropped it before running: the official NRMS is already GloVe-initialised, so the experiment would only have compared our control against a weakened copy of itself.

Control vs. Treatment, All Metrics (Q5)

		AUC	MRR	nDCG@5	nDCG@10	Div.@10	Nov.@10
MIND	Control	0.6831	0.3803	0.3620	0.4279	0.8343	17.623
376,471 imp.	Treatment (+abstract)	**0.6868**	0.3829	0.3691	0.4314	0.8283	17.643
	Paired Δ	+0.0037	+0.0026	+0.0071	+0.0034	**−0.0060**	+0.0197
EB-NeRD	Control	0.5613	0.3512	0.3904	0.4680	0.7894	17.208
244,647 imp.	Treatment (+freshness)	**0.5677**	0.3555	0.3957	0.4725	0.7900	17.236
	Paired Δ	+0.0064	+0.0043	+0.0053	+0.0044	+0.0006	+0.0280

Every paired difference above has a 95% bootstrap CI excluding zero; AUC: MIND $[+0.0031, +0.0044]$, EB-NeRD

$[+0.0053, +0.0075]$; MIND's diversity regression $[-0.0062, -0.0057]$. Coverage@10 moved $0.0627 \rightarrow 0.0626$ (MIND) and $0.2021 \rightarrow 0.1987$ (EB-NeRD), as point estimates only (see Limitations).

Both changes gave gains that were statistically clear but small. EB-NeRD rose from 0.5613 to 0.5677, a paired difference of +0.0064 (95% CI +0.0053 to +0.0075). MIND rose from 0.6831 to 0.6868, a paired difference of +0.0037 (95% CI +0.0031 to +0.0044). MRR, nDCG@5 and nDCG@10 moved the same way on both datasets, with confidence intervals excluding zero. The intervals are narrow because we measured on hundreds of thousands of impressions, and the effects themselves stay small.

We also checked guardrail metrics that should hold steady while AUC improves. EB-NeRD's held: diversity and novelty both rose slightly and coverage barely moved. MIND's did not. Diversity@10 fell by −0.0060 with a confidence interval excluding zero, while novelty improved and coverage stayed flat. We report this as a real cost of the change; the likely reason is that the extra abstract text makes the model more confident about specific topics and narrows the variety of what it ranks highly. The MIND change also cost 2.98 times more compute per training step, which we measured on part of the run before committing to the full training job.

On EB-NeRD the model learned a freshness weight of −0.066, and the negative sign means older articles get pushed down, so the model reached the same conclusion our feature analysis had. Even so, at 0.5677 it remains far behind our LightGBM ranker at 0.7588, so beating the official baseline here did not change which model we submitted.

Slicing the Result (Q5)

Dataset	Slice	n	Paired ΔAUC (95% CI)
MIND	Head (top 20% by train clicks)	143,164	−0.0063 [−0.0071, −0.0053]
MIND	Tail	233,307	+0.0098 [+0.0090, +0.0108]
MIND	Warm users (≥ 5 clicks)	—	+0.0043 [+0.0036, +0.0050]
MIND	Cold users (< 5 clicks)	—	+0.0002 [−0.0020, +0.0022], not significant
EB-NeRD	Head	9,541	−0.0217 [−0.0258, −0.0178]
EB-NeRD	Tail	235,106	+0.0076 [+0.0064, +0.0087]

EB-NeRD has no cold-start users by construction. On the control alone, MIND scores 0.7411 on head against 0.6474 on tail, and 0.6932 on warm users against 0.6209 on cold ones.

Both treatments show the same pattern on two different datasets, with two different changes: the gain comes from tail articles and is diluted by a real loss on popular ones. We did not design for this, and it is the clearest argument for slicing rather than reporting a single average.

5 Q9: Anti-Gaming and Leakage

Some signals exist in the logs but would not be available at serving time, so we measured what they are worth instead of only excluding them. EB-NeRD's lifetime totals (`total_inviews`, `total_pageviews`, `total_read_time`) include activity occurring after an impression; blending them into the deployed embedding score moves AUC from 0.5430 to 0.5662 untuned, which is the advantage we are declining to take. Our loader never reads those columns and a test asserts that reading them raises an error. Two weaker cases were priced the same way: withholding `context_read_time` and `context_scroll_percentage`, known in full only after a user leaves a page, costs 0.0011 AUC (0.7581 against 0.7570), while withholding the publisher's own in-view position *improves* AUC by 0.0007, so our result is not a copy of Ekstra Bladet's existing ranker and the arm we submitted withholds it.

6 Q4: Serving and Scale

The characteristics of the system noted during the run were:

	MIND (CPU)	MIND (GPU)	EB-NeRD
Index / feature store	164.0 MB	164.0 MB	190.5 MB
Mean request	33.29 ms	19.91 ms	9.21 ms
p99 request	**94.67 ms**	**33.15 ms**	**22.74 ms**
Headroom under a 100 ms SLA	1.1×	3.0×	4.4×
Cost per 1,000 queries	$0.0031	$0.0007	$0.0009

MIND's 164.0 MB is the BM25 matrix (23.5 MB), the embedding matrix (110.6 MB) and the NRMS weights (29.9 MB). EB-NeRD's 190.5 MB is mostly its article table (108.2 MB) and user profiles (82.1 MB). All latencies are for a single user request, measured end to end from query to ranked list. The costs assume one worker per instance at AWS on-demand list prices, which is the conservative reading; we avoided assuming perfect scaling across cores because our own profiling showed a 2.5× slowdown from a single concurrent job.

MIND only just clears the 100 ms target on CPU, and its slowest observed request at 126 ms already breaks it. The re-ranker is why: NRMS is 78% of a request on CPU, dropping to 36% on GPU, which is what moves the p99 from 94.67 ms to 33.15 ms. EB-NeRD is comfortable throughout, and its cost sits in feature construction rather than in the model, with LightGBM inference taking only 8% of a request.

There was a difference of roughly 30× noted between EB-NeRD's batched and single-request latency (0.33 ms per impression against about 10 ms for one request), because batching pays the per-call setup once for every 5,000 impressions. The same comparison on MIND gives only about 1.2×, since MIND users average 1.47 impressions each and there is almost nothing to share between them. We report the single-request number as the serving figure, as quoting the batched one would have understated EB-NeRD's real cost thirtyfold. In the training run, the MIND treatment took 34 hours against the control's 13, which was expected: we had measured the extra abstract tokens costing 2.98× per step on part of the run before committing to it.

7 Where It Breaks at 10×

While the system performs well given the current constraints and datasets, there were a few larger scales (10×) where parts of it would suffer.

MIND breaks on latency, not memory. The index grows with the number of articles and not with users, so even a tenfold corpus stays a few hundred megabytes. The problem is that a CPU request already sits at a p99 of 94.67 ms against a 100 ms target, with the re-ranker taking 78% of that time. Moving to GPU is a measured fix rather than a hope, giving 3.0× headroom, and caching the article vectors buys another 7.02× on CPU with no change in accuracy at all. Even together those absorb roughly a 3× increase in traffic, so genuine 10× load needs more workers on top.

EB-NeRD breaks on memory, specifically the user profiles. They cost 5.35 KB per user, which is 82 MB for the 15,342 users in our split but projects to about 4.2 GB at the 791,582 users in the full bundle. Latency is not the constraint here, with 4.4× headroom against the same target.

Scaling up would not improve accuracy on its own. We tested this directly: training the EB-NeRD model on the full bundle, with 10.7× more training data, moved validation AUC from 0.7588 to 0.7590, which is no real change. MIND is already trained at full scale. So 10× is a serving and infrastructure problem for us, and not a route to better scores.

8 Limitations

The shortfalls of our system became visible after testing it in several ways: slicing results by user history length and by article popularity, ablating features individually, running leakage tests against

the real data, and measuring how often retrieval surfaces the clicked article at all. The limitations we encountered were:

- **Retrieval is the real ceiling.** The top 200 contains the clicked article in only about 3% of impressions, so roughly 97% of clicks never reach the re-ranker and no ranking model can recover them.
- **Cold-start users remain unsolved.** On MIND our improvement helped warm users (+0.0043, confidence interval excluding zero) and did nothing measurable for cold ones (+0.0002, not significant). The baseline itself scores 0.6932 on warm users against 0.6209 on cold ones.
- **Both improvements help tail articles and hurt popular ones.** On EB-NeRD the gain splits into −0.0217 on head articles and +0.0076 on tail; on MIND, −0.0063 against +0.0098. The headline gain is a tail effect diluted by a real loss on popular articles.
- **Our baseline is a reimplementation.** We matched the official configuration and reproduce the published number closely, but we never ran the original TensorFlow code.
- **Coverage is reported without a confidence interval.** It counts distinct articles across impressions, and bootstrap resampling repeats users, which adds nothing to that count and would produce a misleadingly narrow interval.
- **MIND's leakage boundary cannot be tested directly.** Its history has no per-click timestamps, so we verify the equivalent property instead, that every history-derived feature traces back to one value per user.
- **Results are bitwise reproducible on CPU and stable to about 8 decimal places on GPU**, which is ordinary CUDA behaviour.

References. C. Wu, F. Wu, S. Ge, T. Qi, Y. Huang, X. Xie. Neural News Recommendation with Multi-Head Self-Attention. *EMNLP-IJCNLP*, 2019. F. Wu et al. MIND: A Large-Scale Dataset for News Recommendation. *ACL*, 2020. J. Kruse et al. EB-NeRD: A Large-Scale Dataset for News Recommendation. *ACM RecSys Challenge*, 2024.